FlashCart — Weekly Project Journal

This journal records our 10 weeks of building FlashCart: what we worked on each week, the problems we hit, and how we fixed them.

Project: FlashCart — Smart Mall Billing and Unified Checkout
Course: UCS503 Software Engineering, Thapar Institute of Engineering and Technology
Team (Group 3C82): Pranay Nagpal (1024031127), Suryansh Jain (1024031145), Srijan Tanwar (1024031144)
Submitted to: Jhonsy Bansal

Weeks 1–6 were iteration 1 (the MVP, shown at the mid-semester evaluation), weeks 7–8 were iteration 2 (accounts, database and cash desk), and weeks 9–10 were iteration 3 (real barcodes and real products).

Week 1 — Understanding the problem

Dates: _____

This week we went to the mall to see how billing really works. We bought a shirt, earbuds and a coffee, stood in three different queues and came back with three separate GST bills. That became our problem statement: one bag and one bill for the whole mall. We listed what a customer and a cashier would need, and chose a web app instead of an Android app so nobody has to install anything.

Problem we faced: We could not agree on how one bill could carry different GST rates. Clothes, electronics and food are taxed differently, but our first idea put a single GST percentage on the whole bill.

How we fixed it: We read about GST and learnt that the rate belongs to each product, through its HSN code. So every product now stores its own GST rate. The bill adds up tax separately for each rate (5%, 12%, 18%) and splits it into CGST and SGST.

What we learnt: Getting the business rules right first saves a redesign later.

Week 2 — Designing the system

Dates: _____

This week was design. We drew the three-tier architecture (React apps, Express API, database, Razorpay), listed the API endpoints, and made the first use case and class diagrams.

Problem we faced: In our first design the phone calculated the bill total and sent that amount to the server to pay. While drawing the sequence diagram we realised anyone could edit that request in the browser and pay Rs. 1 for a Rs. 3,000 shirt.

How we fixed it: We moved all pricing to the server. The app now sends only the barcode and quantity. The server looks up the price and GST and creates the payment order itself. We wrote this down as our main rule: *clients send intents, never amounts*.

Another problem: We were confused whether *Pay online* and *Pay cash* should be «include» or «extend» in the use case diagram. We settled it by asking which steps always happen: the two payment methods became «extend» (the customer picks one), and *Verify payment signature* became «include» (it always happens).

What we learnt: Security has to be part of the design, not added after coding.

Week 3 — Backend and the GST engine

Dates: _____

We set up the Node.js + Express server, the product catalog with barcodes, the bag (session) APIs, and the pricing function `buildBill()` that makes the GST bill.

Problem we faced: Some bill totals were off by one paisa, and the total changed depending on when we rounded. In JavaScript, 0.1 + 0.2 gives 0.30000000000000004, so decimal rupees were adding tiny errors.

How we fixed it: We changed every money calculation to whole paise (integers) and convert to rupees only for display. Now every bill adds up exactly.

Another problem: When the GST came to an odd number of paise (for example 45,747), it could not be split equally into CGST and SGST, and the two halves did not add back to the total. We fixed it by taking CGST = tax ÷ 2 rounded, and SGST = the rest. The two always add up.

Small issue: `node --test test/` did not find our test files. Giving it a pattern, `"test/**/*.test.js"`, fixed it.

What we learnt: Never store or calculate money as decimals.

Week 4 — Building the React frontend

Dates: _____

We built the customer app with React, Vite and Tailwind CSS: the home screen, the catalog, the bag and the checkout page.

Problem we faced: The catalog kept reloading forever. The browser's network tab filled up with hundreds of `/api/products` calls. The function that loaded the catalog was created again on every render, so React's `useEffect` saw a "new" function each time and ran again, in a loop.

How we fixed it: We kept the loaded catalog in a `useRef` and fetch only when it is empty. Now the catalog loads once.

Another problem: After adding an item from the catalog sheet, the Bag tab still said "empty" until we refreshed the page. We fixed it by re-reading the bag from the server whenever the user switches tabs, so the screen always matches the server.

Small issue: After installing a new package, Vite showed "504 Outdated Optimize Dep". Restarting the dev server after installing packages fixed it.

What we learnt: The server is the source of truth, and the screen should re-read it instead of guessing.

Week 5 — Online payments with Razorpay

Dates: _____

This week we connected Razorpay. The server creates an order for the bag's amount, the app opens Razorpay Checkout, and the customer pays by UPI or card.

Problem we faced: How do we know a payment is real? After paying, the browser simply tells our app "success", and that message can be faked.

How we fixed it: Razorpay also returns a signature. Our server recomputes HMAC-SHA256 of `order_id|payment_id` using the secret key, which only the server has, and compares the two in constant time. The bill is saved only if they match; otherwise no invoice is issued.

Another problem: Until our Razorpay test keys were ready, we could not test payments at all. We wrote a small mock gateway for local testing. It signs payments with a temporary secret, so the same verification code still runs.

Small issue: A double tap on *Pay* sent the verify request twice and could make two bills. Now verify is idempotent: a repeat returns the same receipt, and an unchanged bag reuses its open order.

What we learnt: Never trust the browser with money.

Week 6 — First scanner and the mid-semester demo

Dates: _____

We added a camera scanner using the html5-qrcode library and showed the MVP at the mid-semester evaluation.

Problem we faced: In our own testing we had used QR codes, and the scanner worked fine. In the demo we pointed it at real product tags and it either read nothing or picked up the QR code printed on the tag instead of the barcode. Real retail tags use EAN-13 barcodes (the 1D kind with vertical lines), and the library was tuned for QR codes.

What we did on the spot: We switched to *Type barcode* and catalog search, so the demo could continue.

Another problem: We restarted the server once during the demo, and every bag and bill disappeared, because all data was kept only in memory.

How we handled it: We wrote both down as the top issues for the next iteration. The feedback after the demo added two more: separate logins for customers and cashiers, and a cash desk that only deals with cash.

What we learnt: Test with real-world inputs, not only the ones we make ourselves.

Week 7 — Database and two logins

Dates: _____

Iteration 2 started with the two biggest issues from the demo: losing data on restart, and one shared login.

Problem we faced: All bags and bills were lost whenever the server restarted.

How we fixed it: We added a SQLite database using Node's built-in `node:sqlite` module, with tables for customers, cashiers, products, transactions and cash shifts. It is a single file, needs no separate database server, and survives restarts.

Another problem: Every restart inserted the sample products again, so the catalog filled with duplicates. We fixed it with `INSERT OR IGNORE` on the unique barcode, so existing products are skipped.

Third problem: With one shared login, a customer could open the cashier pages. Now customers sign in with just a phone number, and cashiers with email and password. Every login token carries a role, and each part of the API checks it (anyone else gets a 403 error). Cashier passwords are stored as salted scrypt hashes, never as plain text, and wrong guesses are rate-limited (8 tries per 10 minutes).

What we learnt: Logging in (who you are) and permissions (what you may do) are two separate problems.

Week 8 — The cash desk

Dates: _____

This week we built the cashier's screen. The cashier starts the day with the opening cash, finds a bill, enters the cash received, sees the change, and closes the day.

Problem we faced: Our first cash desk also had an item-checking list and an exit pass. It was slow and confusing, and the feedback was that the desk should handle only cash.

How we fixed it: We cut it down to two jobs: how much change to return, and how much cash should be in the drawer at the end of the day (opening cash + cash sales). At closing, the counted cash is compared with that figure to show whether the drawer is over or short.

Another problem: Searching for a bill sometimes matched a food product instead. Food codes started with "FC-", the same prefix as our bag IDs. We renamed the food codes to "FD-".

Small issues: The note breakdown suggested Rs. 2000 notes, which RBI withdrew in 2023, so we removed them; the change now uses notes from Rs. 500 down to Rs. 1. We also blocked the cashier from entering less cash than the bill.

What we learnt: Simpler screens are faster and lead to fewer mistakes.

Week 9 — Replacing the QR scanner with a barcode reader

Dates: _____

Iteration 3 began by fixing the scanner that failed in the mid-semester demo. We removed html5-qrcode and used the browser's built-in `BarcodeDetector` (Chrome). Safari and Firefox don't have it, so on those browsers we fall back to ZXing-C++, a barcode library compiled to WebAssembly.

Problem we faced: The QR code printed on the same tag was still being read instead of the barcode.

How we fixed it: We allowed only barcode formats (EAN-13, EAN-8, UPC, Code 128, Code 39, ITF) and turned QR off completely.

Another problem: Tags held off-centre were missed, but scanning the whole frame every time was slow. Now we scan a wide band in the middle every 90 ms, and the full frame every 4th pass. One read takes 25–35 ms.

Small issues: On phones the camera would not open at `http://<laptop IP>`, because browsers allow the camera only on HTTPS, so we added an HTTPS mode (`npm run dev:https`). One scan sometimes added an item twice, so the same code is now ignored

for 2.2 seconds. And the camera opened straight after login, so we added a home screen first.

What we learnt: Pick a library that fits the real input: shop tags have barcodes, not QR codes.

Week 10 — Real products, testing and final fixes

Dates: _____

In the last week we added the things we had actually bought — a Puma polo (Rs. 2,999), a Mufti shirt (Rs. 2,399) and a JPG parfum — using the barcodes printed on their tags, and tested everything end to end.

Problem we faced: The bill for the polo came to Rs. 3,538.82, more than the Rs. 2,999 printed on the tag. The MRP already includes GST, and our engine was adding 18% on top again.

How we fixed it: We added MRP-inclusive pricing. For tagged items the line total is exactly the MRP, and the taxable value is MRP ÷ 1.18 = Rs. 2,541.53, with GST of Rs. 457.47. A small migration added the new `mrp` column to the existing database without losing data.

Other problems: The load test failed with "out of stock" because a test product had only 10 units, so we used high-stock items. On small phones, buttons were cut off and the cash box ran off the screen, so we fixed the layout and checked it at 375 px width. On college Wi-Fi, the phone could not reach the laptop, so we used the phone's hotspot. Razorpay showed an old remembered phone number, so we now pass in the signed-in customer's number.

Result: All 17 automated tests pass, all 3 real tags scan, and bills match the printed MRP exactly.